ShipShape

Auditing and Improving a Production TypeScript Codebase

Target Repository: US-Department-of-the-Treasury/ship
https://github.com/US-Department-of-the-Treasury/ship

Before You Start: Codebase Orientation (4 Hours)

Before you audit anything, complete the **Appendix: Codebase Orientation Checklist** at the end of this document. This structured process walks you through understanding the repository architecture, the tech stack decisions, the data model, and the development workflow. **Your orientation notes become part of your final submission.**

This week emphasizes reading code over writing code. You will spend more time understanding existing patterns than creating new ones. Orientation is the first step in that methodology.

Background

Every software engineer who joins a new company faces the same challenge on day one: a codebase they have never seen, built by people they have never met, using patterns they may not recognize. The ability to orient yourself in an unfamiliar system, assess its health, and improve it with confidence is the defining skill that separates junior engineers from senior ones.

Ship is a project management tool built by the U.S. Department of the Treasury. It combines documentation, issue tracking, and sprint planning into a single application. The codebase is a TypeScript monorepo with a React frontend, Express backend, PostgreSQL database, and real-time collaboration powered by WebSockets and Yjs. It has 73+ Playwright E2E tests, Docker and Terraform deployment configs, and a unified document model where everything (issues, docs, projects, sprints) lives in a single table.

This project does not ask you to build a new feature from scratch. It asks you to do what production engineers actually do: inherit a system, understand it deeply, measure its health, diagnose its weaknesses, and make it better with proof.

Project Overview

Seven-day sprint with two phases:

Checkpoint	Deadline	Focus
Orientation	4 hours after receiving the project	Read the codebase, complete orientation checklist
Audit Report	Tuesday, 11:59 PM (36 hours)	Baseline measurements for all 7 categories
Implementation	Friday, 11:59 PM	Measurable improvements across all 7 categories
Final Submission	Sunday, 11:59 AM	Polish, documentation, presentation

Phase 1: The Audit (36 Hours)

Hard gate. You must submit a written audit report with baseline measurements for all 7 categories below. This report is your diagnostic. It tells us you understand the system before you touch it.

For each category, you must:

1. Describe how you measured it (tools, commands, methodology)
2. Provide concrete baseline numbers
3. Identify the specific weaknesses or opportunities you found

4. Rank the severity or impact of each finding

You do not fix anything during the audit. *Diagnosis comes before treatment.*

Category 1: Type Safety

What you are measuring: The strength of TypeScript's type system as used in this codebase. This includes explicit **any** types, type assertions (**as**), non-null assertions (**!**), **@ts-ignore** and **@ts-expect-error** directives, untyped function parameters, and implicit **any** from missing return types.

How to Measure

- Run grep or a static analysis tool to count all type safety violations across the codebase
- Check the **tsconfig.json** for strict mode settings. If strict mode is off, run **tsc --strict --noEmit** and count the errors
- Break down violations by package (web/, api/, shared/) and by violation type
- Identify the 5 most violation-dense files and explain why they are problematic

Audit Deliverable

Metric	Your Baseline
Total any types	___
Total type assertions (as)	___
Total non-null assertions (!)	___
Total @ts-ignore / @ts-expect-error	___
Strict mode enabled?	Yes / No
Strict mode error count (if disabled)	___
Top 5 violation-dense files	List with counts

Improvement Target

Eliminate 25% of type safety violations. Every fix must preserve existing functionality (all tests still pass). Superficial fixes do not count. Replacing **any** with **unknown** without proper type narrowing is not an improvement. Each fix must include correct, meaningful types that reflect the actual data.

Category 2: Bundle Size

What you are measuring: The size of the production frontend bundle. Large bundles slow down initial page load, hurt performance on slow networks, and waste bandwidth. You are looking for oversized dependencies, missing code splitting, unused imports, and opportunities to reduce what the browser has to download.

How to Measure

- Build the production frontend and record the total output size
- Use a bundle visualization tool (e.g., **rollup-plugin-visualizer**, **vite-bundle-analyzer**, or **source-map-explorer**) to generate a treemap of the bundle
- Identify the largest chunks and the largest individual dependencies within them

- Check for unused dependencies: cross-reference **package.json** dependencies against actual imports in the source code
- Evaluate whether code splitting is in use and where lazy loading could reduce initial load

Audit Deliverable

Metric	Your Baseline
Total production bundle size	___ KB
Largest chunk	___ (name + size)
Number of chunks	___
Top 3 largest dependencies	List with sizes
Unused dependencies identified	List

Improvement Target

15% reduction in total production bundle size, or implement code splitting that reduces initial page load bundle by 20%. Provide before/after bundle analysis output. Removing functionality to shrink the bundle does not count.

Category 3: API Response Time

What you are measuring: How fast the backend responds under realistic conditions. This is not about testing with an empty database. Seed the database with meaningful volume, then measure.

How to Measure

- Seed the database with realistic data: 500+ documents, 100+ issues, 20+ users, 10+ sprints. Use **pnpm db:seed** or write your own seed script
- Identify the 5 most important API endpoints by tracing the frontend's network requests during common user flows
- Benchmark each endpoint using a load testing tool (**autocannon**, **k6**, **hey**, or similar). Record P50, P95, and P99 response times
- Test under concurrent load: 10, 25, and 50 simultaneous connections
- Identify the slowest endpoints and hypothesize why they are slow

Audit Deliverable

Endpoint	P50	P95	P99
1. ___	___ms	___ms	___ms
2. ___	___ms	___ms	___ms
3. ___	___ms	___ms	___ms
4. ___	___ms	___ms	___ms
5. ___	___ms	___ms	___ms

Improvement Target

20% reduction in P95 response time on at least 2 endpoints. You must provide before/after benchmarks run under identical conditions (same data volume, same concurrency, same hardware). Document the root cause of each bottleneck.

Category 4: Database Query Efficiency

What you are measuring: How efficiently the application queries the database. The unified document model (everything in one table) creates specific query patterns worth examining. You are looking for N+1 queries, missing indexes, full table scans, and unnecessary data fetching.

How to Measure

- Enable PostgreSQL query logging (**log_statement = 'all'** in postgresql.conf or via Docker environment variables)
- Execute 5 common user flows: load the main page, view a document, list issues, load a sprint board, search for content
- Count total queries executed per flow
- Run **EXPLAIN ANALYZE** on the slowest queries
- Check for missing indexes by examining WHERE clauses against existing indexes
- Identify N+1 patterns: places where a list view triggers one query per item instead of a batch query

Audit DeliverableImprovement Target

User Flow	Total Queries	Slowest Query (ms)	N+1 Detected?
Load main page	___	___ms	Yes / No
View a document	___	___ms	Yes / No
List issues	___	___ms	Yes / No
Load sprint board	___	___ms	Yes / No
Search content	___	___ms	Yes / No

20% reduction in total query count on at least one user flow, or 50% improvement on the slowest query. Provide before/after **EXPLAIN ANALYZE** output. Document what was inefficient and why your change fixes it.

Category 5: Test Coverage and Quality

What you are measuring: What the existing test suite covers, what it misses, and how reliable it is. Ship has 73+ Playwright E2E tests. Your job is to understand what they test, find the gaps, and assess test reliability.

How to Measure

- Run the full test suite: **pnpm test**. Record pass/fail counts and total runtime
- Read the test files. Catalog what user flows are covered and which are not
- Identify flaky tests: run the suite 3 times and note any tests that pass sometimes and fail others
- Map critical user flows (document CRUD, real-time sync, auth, sprint management) against existing test coverage

- If code coverage tooling is not configured, configure it and report line/branch coverage per package

Audit Deliverable

Metric	Your Baseline
Total tests	___
Pass / Fail / Flaky	___ / ___ / ___
Suite runtime	___s
Critical flows with zero coverage	List them
Code coverage % (if measured)	web: ___% / api: ___%

Improvement Target

Add meaningful tests for 3 previously untested critical paths, or fix 3 flaky tests with documented root cause analysis. "Meaningful" means the test catches a real regression, not just asserting that a page loads. Each test must include a comment explaining what risk it mitigates.

Category 6: Runtime Error and Edge Case Handling

What you are measuring: How the application behaves when things go wrong. This covers error boundaries, unhandled promise rejections, network failure recovery (especially during real-time collaboration), malformed input handling, and user-facing error states.

How to Measure

- Open browser DevTools and monitor the console during normal usage. Count errors and warnings
- Test network failure: disconnect while editing a document collaboratively, then reconnect. Does data survive? Does the UI recover?
- Test malformed input: submit empty forms, extremely long text, special characters, HTML/script injection
- Test concurrent edge cases: two users editing the same document field simultaneously
- Throttle the network to 3G and use the app. Note every spinner that hangs, every silent failure, every missing loading state
- Check server logs for unhandled errors during all of the above

Audit Deliverable

Metric	Your Baseline
Console errors during normal usage	___
Unhandled promise rejections (server)	___
Network disconnect recovery	Pass / Partial / Fail
Missing error boundaries	List locations
Silent failures identified	List with reproduction steps

Improvement Target

Fix 3 error handling gaps. At least one must involve a real user-facing data loss or confusion scenario (not just a missing loading spinner). Each fix requires reproduction steps, before/after behavior, and a screenshot or recording.

Category 7: Accessibility Compliance

What you are measuring: Ship claims Section 508 compliance and WCAG 2.1 AA conformance. Your job is to verify those claims. This means automated accessibility scanning, keyboard navigation testing, screen reader testing, and color contrast verification across the application's major pages.

How to Measure

- Run Lighthouse accessibility audits on every major page of the application. Record the score for each
- Run an automated accessibility scanner (**axe-core**, **pa11y**, or the axe browser extension) and categorize violations by severity (Critical, Serious, Moderate, Minor)
- Test full keyboard navigation: can you reach every interactive element using only Tab, Enter, Escape, and arrow keys?
- Test with a screen reader (VoiceOver, NVDA, or similar). Can you understand the page structure and interact with all controls?
- Check color contrast ratios on text, buttons, and interactive elements against the WCAG 2.1 AA 4.5:1 minimum

Audit Deliverable

Metric	Your Baseline
Lighthouse accessibility score (per page)	List scores
Total Critical/Serious violations	___
Keyboard navigation completeness	Full / Partial / Broken
Color contrast failures	___
Missing ARIA labels or roles	List locations

Improvement Target

Achieve a Lighthouse accessibility score improvement of 10+ points on the lowest-scoring page, or fix all Critical/Serious violations on the 3 most important pages. Provide before/after Lighthouse reports or axe scan output as evidence.

Category 8: Terraform Plan Review

What you are measuring: Whether you can read and reason about infrastructure-as-code. This is not a writing exercise — the Terraform already exists in the repo. Your job is to understand it, run it locally using the Terraform local provider, and demonstrate you can identify what a plan will change and what the blast radius is. Deployment is done via Render, which has an official first-party Terraform provider. No AWS account or cloud credentials are required.

How to Measure

- Install Terraform locally. The local exercises use the Terraform local provider (hashicorp/local) — no cloud account needed. For deployment, configure the Render Terraform provider (registry.terraform.io/render-oss/render) with a Render API key. Navigate to terraform/ and run terraform init followed by terraform plan. Save the full plan output.

- Annotate the plan: for every resource it will create, modify, or destroy, write one sentence explaining what it is and whether the change is safe. Flag any resource change you consider risky and explain why.
- Simulate drift: using the local provider, write a Terraform config that manages a local file resource. Manually edit the file outside of Terraform to simulate a drift condition. Re-run terraform plan and capture the diff showing what Terraform detects has changed. For the Render deployment, you can simulate drift by changing a setting in the Render dashboard, then running terraform plan to show Terraform detecting the inconsistency.
- Identify the blast radius: if terraform apply were run right now, what is the worst-case impact? Which resources would be recreated (causing downtime), which would be modified in place, and which are safe no-ops?

Audit Deliverable

- Annotated terraform plan output explaining every resource and its blast radius
- Drift detection demonstration: before-and-after plan output showing a manual change being detected

Improvement Target

Write a new Terraform config that uses the local provider to manage at least two local resources (e.g. configuration files, environment files). Then write a second config using the Render provider that declares a Render web service and deploys your improved ShipShape fork. Both configs must have pinned provider versions. Run terraform plan on each and confirm the output matches intent. The Render deployment replaces any manual deploy steps — your fork should be deployable from a clean machine using only terraform apply.

Phase 2: Implementation (4.5 Days)

Improve all 8 categories. Your audit report guides your priorities, but you must deliver measurable improvement in every category. The passing threshold for each category is defined by its Improvement Target above.

Implementation Rules

1. **Before/After proof is mandatory.** Every improvement must include a reproducible benchmark or measurement showing the before state and the after state, run under identical conditions.
2. **Tests must still pass.** If any existing test breaks because of your change, you must either fix the test (with justification) or revert the change. Any change that causes a regression in the CI pipeline must be rolled back immediately — do not merge a PR that breaks the build or test suite.
3. **Regression tests are required.** Every bug or vulnerability found during the audit must have a corresponding regression test that would have caught it. Tests that mock external services must use stable fakes (not live external calls) so they pass consistently in CI regardless of network conditions.
4. **CI pipeline required.** Add GitHub Actions workflows that run on every PR and commit: build, lint, type-check, test, coverage, dependency audit (pnpm audit), and security scan. All checks must pass before a PR can merge. Dependency versions must be pinned in package.json and lockfiles committed. Produce a source-code inventory as part of the CI run — a list of all packages, their versions, and their license. Any deviation from required checks must be documented with written justification.
5. **Build/release/run separation.** Build artifacts (compiled output, Docker images) must be produced once and promoted through environments — never rebuilt per environment. The artifact produced in CI must be the artifact that runs in production. Tag each artifact with the git commit SHA for provenance. Document the artifact lifecycle in your dev docs.

6. **One-command local start.** Write a script (e.g. ./start.sh or a Makefile target) that starts the full composed system locally — app, database, and any mock external services — with a single command from a clean checkout. This script must be documented in the README cold-start guide and must work without any manual setup steps beyond installing dependencies.
7. **Retries, timeouts, and circuit breakers.** Assess the existing codebase for missing retry logic, hardcoded timeouts, and missing circuit breaker patterns on outbound service calls (database, WebSocket, external APIs). Add or improve these where gaps are found. Document each change with the failure mode it protects against.
8. **Dev documentation required.** Every addition or improvement you make must be accompanied by developer documentation. This is separate from the audit report and improvement documentation — it is written for the next engineer who inherits this codebase. At minimum: what was added, how to run it, how to test it, and how to roll it back if it breaks. Store this in a CHANGES.md file at the repo root.
9. **Document your reasoning.** For each improvement, write a short explanation of: what you changed, why the original code was suboptimal, why your approach is better, and what tradeoffs you made.
10. **No cosmetic changes.** Renaming variables, reformatting code, or updating comments do not count as improvements unless they directly support a measurable change in one of the 7 categories.
11. **Commit discipline matters.** Each improvement should be in its own branch or clearly separated commit(s) with descriptive messages. We will read your git history.

Learning Objectives

This project is designed to build the skills that matter most in your first 90 days at a new job.

Skill	How This Project Develops It
TypeScript Fluency	You must read and understand thousands of lines of TypeScript before changing a single one. The type safety audit forces you to understand generics, discriminated unions, utility types, and strict mode.
Software Architecture	You are analyzing a real monorepo with clear separation of concerns (web/, api/, shared/), a unified document model, and real-time collaboration infrastructure. You must understand how these pieces fit together.
System Architecture	The full stack includes PostgreSQL, Express, React, WebSockets, Yjs, Docker, and Terraform. You will trace data flow from the browser through the API to the database and back.
Codebase Navigation	Day-one skill at every job. You will develop a systematic process for orienting yourself in an unfamiliar codebase: finding entry points, tracing request flows, reading documentation.
Performance Engineering	Profiling, load testing, query analysis, build optimization, bundle analysis. You will learn to measure before you optimize, and prove your improvements with data.
Professional Engineering Practices	Commit discipline, before/after benchmarking, written technical reasoning, reproducible measurements. This is how senior engineers work.

Discovery Requirement

Find 3 things in this codebase that you did not know before. These can be TypeScript features, architectural patterns, libraries, design decisions, or engineering practices that were new to you. For each discovery:

1. Name the thing you discovered
2. Where you found it in the codebase (file path and line range)
3. What it does and why it matters
4. How you would apply this knowledge in a future project

Ship Tech Stack Reference

You are working with this codebase, not choosing your own stack. Understand it deeply.

Layer	Technology	Key Files
Frontend	React, Vite, TailwindCSS	web/src/
Editor	TipTap + Yjs (real-time collaboration)	web/src/ (editor components)
Backend	Express, Node.js	api/src/
Database	PostgreSQL	api/src/db/
Real-time	WebSocket + Yjs	api/src/collaboration/
Shared Types	TypeScript	shared/
Testing	Playwright E2E	e2e/
Infrastructure	Docker, Terraform	Dockerfile, terraform/
Package Manager	pnpm workspaces	pnpm-workspace.yaml

Key Architecture Decisions

Everything is a document. Issues, wiki pages, projects, and sprints all share a single documents table with a document_type discriminator. Understand the tradeoffs of this approach.

Server is truth. The app is offline-tolerant but server-authoritative. Real-time sync uses Yjs CRDTs for the editor and WebSocket messages for presence and cursor tracking.

Boring technology. The team deliberately chose well-understood tools. When you find something that could be "better" with a newer tool, ask yourself what the original team valued and why.

Submission Requirements

Deliverable	Requirements
GitLab Repository	Forked repo with all improvements on clearly labeled branches. Setup guide in README.
Audit Report	Written report with baseline measurements for all 7 categories. Include methodology, tools used, and raw data.

Improvement Documentation	For each of the 7 categories: before measurement, explanation of root cause, description of fix, after measurement, proof of reproducibility.
Discovery Write-up	3 things you learned, with codebase references and reflection.
Demo Video (3-5 min)	Walk through your audit findings and improvements. Show before/after measurements. Explain your reasoning.
AI Cost Analysis	Dev spend + reflection on AI tool effectiveness for codebase comprehension.
Deployed Application	Your improved fork running and publicly accessible.
Social Post	Share on X or LinkedIn: what you learned auditing a government codebase, key findings, tag @GauntletAI.
Terraform Plan Review	Terraform config using the local provider (local resources) and the Render provider (web service deployment). Annotated terraform plan output with blast radius analysis. Drift detection demonstration using the local provider. Provider versions pinned in all modules. Render deployment verifiable via terraform apply from a clean checkout.

How This Is Graded

Audit Report (Pass/Fail Gate)

Your audit report must include baseline measurements for all 7 categories. Incomplete audits are an automatic fail regardless of implementation quality. We are assessing your ability to diagnose a system, not just fix one.

Implementation (Scored)

Criteria	Weight	What We Look For
Measurable improvement	40%	Did you hit the target in all 7 categories? Are your before/after measurements reproducible?
Technical depth	25%	Do your fixes demonstrate genuine understanding of the root cause, or are they surface-level patches?
TypeScript quality	15%	Is your new code well-typed? Do you use TypeScript features appropriately (generics, narrowing, utility types)?
Documentation quality	10%	Is your reasoning clear, concise, and technically sound? Could another engineer follow your logic?
Commit discipline	10%	Clean git history, descriptive messages, logical separation of changes.

Final Note

A thorough audit with targeted, well-documented improvements beats a scattered attempt to fix everything superficially. Depth over breadth. Proof over promises.

Appendix: Codebase Orientation Checklist

Complete this before auditing. Save your notes as a reference document. The goal is to build a mental model of the entire system before measuring anything.

Phase 1: First Contact

1. Repository Overview

- Clone the repo and get it running locally. Document every step, including anything that was not in the README.
- Read every file in the docs/ folder. Summarize the key architectural decisions in your own words.
- Read the shared/ package. What types are defined? How are they used across the frontend and backend?
- Create a diagram of how the web/, api/, and shared/ packages relate to each other.

2. Data Model

- Find the database schema (migrations or seed files). Map out the tables and their relationships.
- Understand the unified document model: how does one table serve docs, issues, projects, and sprints?
- What is the document_type discriminator? How is it used in queries?
- How does the application handle document relationships (linking, parent-child, project membership)?

3. Request Flow

- Pick one user action (e.g., creating an issue) and trace it from the React component through the API route to the database query and back.
- Identify the middleware chain: what runs before every API request?
- How does authentication work? What happens to an unauthenticated request?

Phase 2: Deep Dive

4. Real-time Collaboration

- How does the WebSocket connection get established?
- How does Yjs sync document state between users?
- What happens when two users edit the same document at the same time?
- How does the server persist Yjs state?

5. TypeScript Patterns

- What TypeScript version is the project using?
- What are the tsconfig.json settings? Is strict mode on?
- How are types shared between frontend and backend (the shared/ package)?
- Find examples of: generics, discriminated unions, utility types (Partial, Pick, Omit), and type guards in the codebase.
- Are there any patterns you do not recognize? Research them.

6. Testing Infrastructure

- How are the Playwright tests structured? What fixtures are used?
- How does the test database get set up and torn down?
- Run the full test suite. How long does it take? Do all tests pass?

7. Build and Deploy

- Read the Dockerfile. What does the build process produce?
- Read the docker-compose.yml. What services does it start?
- Read the Terraform configs in terraform/. Understand what infrastructure the app declares. Note which provider is used and whether versions are pinned. In Week 4 you will replace or extend this config using the Terraform local provider for local exercises and the Render provider (render-oss/render) for deployment — no AWS account is needed.
- How does the CI/CD pipeline work (if configured)?

Phase 3: Synthesis

8. Architecture Assessment

- What are the 3 strongest architectural decisions in this codebase? Why?
- What are the 3 weakest points? Where would you focus improvement?
- If you had to onboard a new engineer to this codebase, what would you tell them first?
- What would break first if this app had 10x more users?